

Informe Técnico — Sprint 3

Sistema de Predicción de Retrasos Logísticos — Olist

Selección de Modelo, Optimización e Hiperparametrización

Resumen Ejecutivo

Este sprint tuvo como objetivo seleccionar el modelo óptimo para predecir si un pedido de Olist llegará tarde, optimizar sus hiperparámetros y exportar el modelo final reproducible.

Modelo seleccionado: Regresión Logística Tuneada

AUC-ROC Validación: 0.7532

AUC-ROC Backtest: 0.7375

Recall con umbral 0.30: 96.5%

Archivo exportado: `models/champion_model_v3.pkl`

1. Contexto y Datos

Dataset

- Fuente:** Brazilian E-Commerce Public Dataset by Olist (Kaggle)
- Período:** Septiembre 2016 - Julio 2018 (agosto 2018 excluido por censura de datos)
- Pedidos analizados:** 89,802 (filtrados desde 96,470 originales)
- Variable objetivo:** `is_late_delivery` — 1 si llegó después de la fecha prometida

Filtros aplicados antes del modelado

Filtro	Registros eliminados	Razón
Sin fecha de entrega	0	Todos tenían fecha
Post 31-Jul-2018 (agosto)	6,664	Censura de datos — target inválido
Días estimados fuera de rango	4	Errores de captura
Total eliminados	6,668	

¿Por qué se eliminó agosto? El dataset fue extraído el 29-Ago-2018. Los pedidos realizados en agosto con entrega estimada posterior a esa fecha aparecen como "a tiempo" cuando en realidad su resultado era desconocido. Incluirlos contaminaría el target con ceros artificiales.

Distribución de particiones temporales

Partición	Período	Pedidos	Tasa tardíos	Uso
Train	Sep 2016 – Mar 2018	64,152	9.1%	Entrenamiento
Val	Abr – May 2018	13,415	6.8%	Calibración de umbral
Backtest	Jun 2018	5,973	1.4%	Evaluación de estabilidad
Live	Jul 2018	5,843	4.1%	Simulación de producción

Justificación de los cortes: Train incluye los meses de mayor estrés logístico (Feb 2018: 16%, Mar 2018: 21%) para que el modelo aprenda patrones de riesgo alto. Val con dos meses tiene suficientes positivos para calibrar el umbral. Backtest de junio sirve para medir estabilidad. Live de julio es la evaluación final.

2. Variable Objetivo

```
is_late_delivery = 1 → order_delivered_customer_date >
order_estimated_delivery_date
is_late_delivery = 0 → Entregado a tiempo o antes
```

Distribución: 7.9% de tardíos (7,130 de 89,802 pedidos)
Desbalance: ~1:10.5 (positivos:negativos)
Decisión: Variable binaria porque el negocio necesita una decisión de intervención, no una predicción de días exactos.

3. Feature Engineering — 50 Variables Finales

Las 50 variables se construyeron en 6 grupos, cada uno vinculado a las hipótesis del Sprint 1:

Grupo 1 — Variables Temporales (15)

Vinculadas a H3 (el día y hora de compra impactan el resultado).
Variables clave: `purchase_weekday`, `purchase_hour`, `high_risk_day`, `is_peak_season`, `estimated_delivery_days`, `is_short_promise`, `approval_delay_flag`

Grupo 2 — Variables Geográficas (6)

Vinculadas a H1 (la complejidad logística, no la distancia sola).

Variables clave: `region_risk_score`, `same_state`, `complex_route`, `distance_km`, `is_long_distance`

Grupo 3 — Variables de Producto (10)

Vinculadas a H1 y H4 (complejidad del producto define el riesgo).

Variables clave: `logistic_complexity`, `volume_cm3`, `has_oversized_item`, `carrito_complejo`, `tiene_varios_vendedores`

Grupo 4 — Variables de Pago (8)

Vinculadas a H4 (el método de pago agrega latencia al proceso).

Variables clave: `freight_ratio`, `is_boleto`, `is_high_installments`, `freight_per_kg`

Grupo 5 — Historial del Vendedor (3)

Sin data leakage: rolling expanding window — solo usa información pasada.

Variables clave: `seller_late_rate_hist`, `seller_total_orders_hist`, `seller_is_experienced`

Grupo 6 — Score de Riesgo Compuesto (3)

Vinculado a H4 (existen perfiles de riesgo diferenciados).

Variables clave: `risk_score` (0-10), `region_x_complexity`, `distance_x_complexity`

4. Selección de Variables

Pipeline de 3 pasos aplicados sobre datos crudos (sin WOE ni scaling):

Paso	Criterio	Umbral	Variables eliminadas	Variables restantes
0	Inicial	—	—	67
1	Missing rate	> 10%	0	67
2	PSI (estabilidad)	> 0.15	4	63
3	Correlación	> 0.90	13	50

¿Por qué PSI = 0.15? Umbral estándar de la industria financiera. Se calcula sobre datos crudos para evitar que las transformaciones (WOE, scaling) inflen artificialmente el índice. Variables eliminadas por PSI alto son inestables entre el período de entrenamiento y validación.

¿Por qué no se usaron IV ni RF importancia? Son filtros agresivos que eliminan variables con lógica de negocio válida pero baja correlación univariante con el target. Se decidió mantener todas las variables que pasan los 3 filtros estadísticos básicos.

5. Manejo del Desbalance

Técnicas evaluadas (4 técnicas × 2 modelos = 10 combinaciones)

Técnica	LogReg AUC Val	LightGBM AUC Val	Ganador
sin_balanceo	0.7528	0.7328	LogReg
RandomOverSampler	0.7523	0.7307	—
SMOTE	0.6645	0.7356	LightGBM
SMOTENC	0.6863	0.7304	—
SMOTEENN	0.6734	0.7312	—

Conclusión LogReg: `class_weight='balanced'` es suficiente. SMOTE perjudica porque la regresión logística con frontera lineal se confunde con los ejemplos sintéticos generados en espacios que no siguen la geometría lineal del modelo.

Conclusión LightGBM: SMOTE reduce el gap de sobreajuste de 0.172 a 0.120 porque fuerza al modelo a aprender mejor los patrones de la clase minoritaria.

6. Modelos Evaluados

Parámetros base (sin tuneo)

Modelo	AUC Train	AUC Val	Gap	Recall Val	Estado
Regresión Logística	0.7166	0.7528	-0.036	73.8%	✅ Sin sobreajuste
Random Forest	1.0000	0.7145	0.285	6.0%	❌ Sobreajuste total
LightGBM	0.9049	0.7311	0.172	61.5%	⚠️ Sobreajuste moderado
XGBoost	0.9806	0.6875	0.293	46.0%	❌ Sobreajuste severo

Por qué Random Forest tiene AUC Train = 1.0: Con `max_depth=None` los árboles crecen hasta memorizar cada pedido del entrenamiento. No aprende patrones generalizables.

Por qué XGBoost y LightGBM sobreajustan: Los gradient boosting con parámetros default y `(n_estimators=200)` sin early stopping memorizan el ruido de entrenamiento. Necesitan regularización explícita.

7. Optimización con Optuna

Configuración

- **Herramienta:** Optuna con TPESampler (búsqueda bayesiana)
- **Trials:** 200 por modelo
- **Validación interna:** StratifiedKFold con 5 folds sobre train
- **Función objetivo:** `AUC_CV - max(0, gap - 0.10) × 0.5`
- **Penalización:** Si el gap Train→Val supera 0.10, se descuenta del score. Esto fuerza a Optuna a buscar modelos generalizables, no solo con alto AUC puntual.

Por qué CV=5 en lugar de evaluar directo en Val

Evaluar directo en Val (como se hizo en la versión anterior) causa **overfitting al conjunto de validación**: después de 200 trials Optuna aprende las particularidades de esos 13,415 pedidos específicos. Con CV=5 el score es el promedio de 5 evaluaciones en particiones distintas de train — más robusto y honesto.

Hiperparámetros tuneados — LogReg

Parámetro	Base	Tuneado	Cambio
C	1.0	0.1731	Más regularización 
solver	lbfgs	liblinear	Cambió
penalty	l2	l1	Cambió a L1 (sparsity)

Interpretación C: C controla la regularización inversa en LogReg. Un C menor significa más regularización — el modelo es más conservador y menos propenso a memorizar. El valor base de 1.0 era insuficiente; 0.1731 encontrado por Optuna produce un modelo más generalizable.

Hiperparámetros tuneados — LightGBM

Parámetro	Base	Tuneado	Impacto
n_estimators	200	389	Más árboles
num_leaves	31	37	Levemente más complejo

Parámetro	Base	Tuneado	Impacto
learning_rate	0.1	0.022	Más lento → mejor generalización
min_child_samples	20	50	Más regularización
reg_alpha	0.0	0.684	L1 añadida
reg_lambda	0.0	0.159	L2 añadida
feature_fraction	1.0	0.619	Subsampling
bagging_fraction	1.0	0.921	Bagging

8. Resultados Post-Tuneo

Comparativa base vs tuneado — Validación

Modelo	AUC Val	Gap	Recall	F1	Estado
LogReg BASE	0.7528	-0.036	73.8%	0.221	
LogReg TUNEADO	0.7532	-0.037	96.5%	0.149	Champion
LightGBM BASE	0.7311	0.172	61.5%	0.234	
LightGBM TUNEADO	0.7444	0.234	2.7%	0.050	

Nota sobre LightGBM tuneado: El Recall colapsó a 2.7% porque la regularización excesiva junto con el umbral 0.5 hizo al modelo demasiado conservador. La penalización del gap resolvió el sobreajuste pero creó el problema opuesto: el modelo predice casi todo como "a tiempo". Con umbral calibrado este problema se corregiría, pero dada la inestabilidad general de LightGBM en este dataset, se mantiene LogReg como champion.

9. Calibración del Umbral

El umbral de decisión es crítico para este negocio. Se evaluaron dos estrategias:

Estrategia	Umbral	Recall	Precisión	F1	Interpretación
Maximizar F1	0.66	42.0%	19.9%	0.270	Balance recall/precisión
Maximizar Recall	0.30	96.5%	8.1%	0.149	Detectar casi todo

Decisión: umbral 0.30 — Para un sistema de predicción de retrasos logísticos, la métrica más importante es el Recall. Un tardío no detectado significa un cliente insatisfecho sin intervención preventiva. Una falsa alarma significa solo una intervención innecesaria (costo: R\$8). La ecuación de negocio favorece detectar el máximo de tardíos posibles.

10. Backtesting Temporal

Período	Pedidos	Tardíos	AUC-ROC	Recall	F1	Tasa Detección
Val (Abr-May 18)	13,415	6.8%	0.7532	96.5%	0.149	96.5%
Backtest (Jun 18)	5,973	1.4%	0.7375	91.6%	0.038	91.6%
Live (Jul 18)	5,843	4.1%	0.6592	95.8%	0.084	95.8%

Estabilidad: El Recall se mantiene por encima del 90% en los 3 períodos. El F1 bajo en Backtest se explica por la tasa de tardíos de solo 1.4% (83 positivos en 5,973 pedidos) — con tan pocos positivos el F1 es naturalmente bajo aunque el Recall sea alto.

Caída del AUC en Live (0.6592): Julio 2018 tiene la promesa media más baja del dataset (19.6 días vs 22–25 días en el período de entrenamiento). Esta variable es la sexta más importante del modelo, y su cambio de distribución genera drift en la predicción. Se abordará en el Sprint 4 con validación cruzada temporal.

11. Importancia de Variables del Modelo Champion

Ranking	Variable	Importancia	Hipótesis
1	tiene_varios_vendedores	1.1935	H4
2	seller_state	0.9373	H1
3	customer_state	0.8734	H1
4	main_category	0.8364	H1
5	same_state	0.6489	H1
6	estimated_delivery_days	0.5871	H2
7	is_short_promise	0.5299	H2
8	is_festivo	0.5103	H3

Ranking	Variable	Importancia	Hipótesis
9	log_total_payment	0.4178	H4
10	complex_route	0.4047	H1

Las 4 hipótesis del Sprint 1 tienen respaldo empírico:

- **H1 (complejidad > distancia):** (seller_state), (customer_state), (same_state) y (complex_route) están en el top 10. La distancia pura ((is_long_distance)) tiene menor peso.
- **H2 (promesa irreal):** (estimated_delivery_days) e (is_short_promise) en posiciones 6 y 7.
- **H3 (factor calendario):** (is_festivo) en posición 8, (is_peak_season) en top 12.
- **H4 (perfiles de riesgo):** (tiene_varios_vendedores) es la variable #1 — los pedidos multi-vendedor concentran el riesgo de coordinación.

12. Archivos del Modelo Final

Archivo	Descripción
(models/champion_model_v3.pkl)	Modelo serializado con todos los artefactos
(models/champion_resumen_v3.json)	Metadatos, parámetros y métricas
(data/features_finales.pkl)	Lista de 50 features del modelo
(data/transformaciones.pkl)	WOE maps + Scaler + estrategia imputación
(data/metricas_modelo_final.csv)	Métricas completas por split
(data/variables_finales_importancia.csv)	50 variables + PSI + importancia
(data/tabla_comparativa_tuneo.csv)	Base vs tuneado
(data/tabla_hiperparametros.csv)	Cambios en cada hiperparámetro
(data/tabla_balanceo.csv)	4 técnicas × 2 modelos
(data/tabla_backtesting_tuneado.csv)	Val / Backtest / Live
(data/optuna_historial_logreg.csv)	200 trials LogReg
(data/optuna_historial_lightgbm.csv)	200 trials LightGBM
(reports/dashboard_sprint3_completo.png)	Dashboard completo de métricas

Archivo	Descripción
reports/seleccion_modelo.png	Justificación visual de la selección
reports/curva_roc_tuneado.png	ROC base vs tuneado
reports/optuna_historial.png	Convergencia de Optuna
reports/importancia_variables_champion.png	Feature importance
reports/comparativa_balanceo.png	Técnicas de balanceo

13. Cómo Reproducir el Modelo

```
bash

# 1. Instalar dependencias
pip install pandas numpy sqlalchemy psycpg2-binary scikit-learn \
    category-encoders pyarrow joblib lightgbm xgboost \
    optuna imbalanced-learn shap streamlit plotly pillow

# 2. Configurar contraseña PostgreSQL en 02_feature_engineering.py

# 3. Ejecutar pipeline completo
python 02_feature_engineering.py      # → master_table_dirty.parquet
python 03_preparar_datos.py          # → split_*.parquet
python 04_seleccion_variables.py      # → features_finales.pkl
python 05_modelado.py                # → champion_model_v2.pkl
python 06_tuneo.py                   # → champion_model_v3.pkl (FINAL)
python 07_reporte_sprint3.py         # → verificación y gráficos

# 4. Dashboard
streamlit run app.py
```

Semilla fija: `SEED = 42` en todos los scripts.

Verificación de reproducibilidad: `07_reporte_sprint3.py` compara AUC guardado en pkl vs AUC recalculado. Diferencia esperada: < 0.001.

14. Conclusión

La Regresión Logística Tuneada es el modelo champion por tres razones fundamentales:

Técnica: Es el único modelo sin sobreajuste (gap = -0.037). Los modelos basados en árboles (Random Forest, XGBoost, LightGBM) sobreajustan severamente con parámetros default y

requieren regularización más agresiva de la que Optuna pudo encontrar en 200 trials con el espacio de búsqueda disponible.

De negocio: Con umbral 0.30 detecta el 96.5% de los pedidos tardíos de forma consistente en los 3 períodos de evaluación. En logística, un tardío no detectado es un cliente insatisfecho sin posibilidad de intervención preventiva.

De producción: Es interpretable (los coeficientes son directamente la importancia de cada variable), rápido (milisegundos por predicción), y estable temporalmente (Val→Back→Live sin colapsos).

Si bien metodológicamente LightGBM tiene más capacidad teórica para capturar relaciones no lineales, en este dataset específico — con 50 features WOE-encoded que ya transforman las relaciones a un espacio más lineal — la regresión logística captura los patrones relevantes sin sobreajustar.

*Proyecto: Sistema de Inteligencia Logística — Olist E-commerce
Sprint 3 — Selección de Modelo, Optimización e Hiperparametrización*